

Мы сосредоточились на использовании Docker как инструмента для решения различных типов проблем. Тем временем мы решили отложить некоторые проблемы до потом и полностью игнорировать другие.

Цель этой части — рассмотреть некоторые из лучших практик контейнеров и улучшить наши процессы.

В [части 1](#) мы упоминали, как [Alpine Linux](#) образ намного меньше Ubuntu, но мы не вдавались в объяснение, почему мы можем выбрать один над другим.

Помимо этого, мы запускали приложения от имени [root](#), то есть суперпользователя, что потенциально опасно.

Изучение официальных образов

Какая версия считается "официальной", решают мейнтейнеры [Docker Official Images](#). [Репозиторий официальных образов](#) содержит библиотеку образов, считающихся официальными. Они вводятся в библиотеку через обычный процесс pull request. Расширенный процесс проверки образа описан в README репозитория.

Многие из наиболее известных проектов поддерживаются под организацией [docker-library](#). К ним относятся образы, такие как Postgres и Python. Однако многие из них включены в библиотеку, но управляются отдельной организацией, такой как Ubuntu и [Node.js](#)

Давайте посмотрим на образ Ubuntu на [Docker Hub](#) и отследим процесс.

Описание/readme говорит:

Что в этом образе?

Этот образ собран из официальных rootfs tarballs, предоставленных Canonical (см. dist-* tags на <https://git.launchpad.net/cloud-images/+oci/ubuntu-base>).

Мы видим, что образ собран из <https://git.launchpad.net/cloud-images/+oci/ubuntu-base>.

Давайте внимательнее посмотрим на Ubuntu, чтобы проверить, откуда он берется. Если вы нажмете ссылку Dockerfile на https://hub.docker.com/r/_ubuntu/, вам будет предоставлен json файл вместо содержимого Dockerfile. Дайджест, похоже, содержит валидный дайджест для версии с одной архитектурой, перечисленной на Docker Hub (amd64). Но после загрузки и проверки дайджеста `docker pull ubuntu:22.04 && docker image ls --digests` результаты не совпадают.

Мы находим Dockerfile из репозитория [здесь](#). Мы можем увеличить доверие к тому, что это тот же образ, который мы загрузили, с помощью `image history` :

```
$ docker image history --no-trunc ubuntu:22.04
```

Вывод из image history совпадает с директивами, указанными в Dockerfile. Если этого недостаточно, мы также могли бы собрать образ сами.

Первая строка говорит, что образ начинается FROM **специального** образа "scratch", который просто пуст. Затем файл `ubuntu-*oci-$LAUNCHPAD_BUILD_ARCH-root.tar.gz` добавляется в корень из той же [директории](#).

Обратите внимание, как файл не извлекается ни в какой момент. Документация инструкции `ADD` [утверждает](#), что "Если `src` — это локальный `tar` архив в распознаваемом формате сжатия (`identity`, `gzip`, `bzip2` или `xz`), то он распаковывается как директория."

Мы могли бы проверить контрольные суммы файла, если бы нам было интересно. Для образа Ubuntu автоматизация из `launchpad` заботится о создании PR в `docker-library`, и мейнтейнеры репозитория официальных образов проверяют PR.

Вы также можете посетить страницу Docker Hub для самого тега образа, которая показывает слои и предупреждает о потенциальных проблемах безопасности. Вы можете увидеть, сколько различных проблем она находит [здесь](#).

Теперь мы узнали, что процессы сборки открыты, и мы можем проверить их, если у нас есть такая необходимость. Кроме того, мы узнали, что нет ничего особенного в "официальных" образах.

"Вы не можете доверять коду, который вы не создали полностью сами."

- КЕН ТОМПСОН (1984, РАЗМЫШЛЕНИЯ О ДОВЕРИИ)